

1. Dada una base de datos con varias tablas, ¿cuál(es) de las siguientes restricciones se puede(n) usar para asegurar, o para definir que no se permitirá la inserción de valores NULL?

- UNIQUE
- CHECK NOT
- NULL (NOT)
- FOREIGN KEY
- PRIMARY KEY

2. ¿Las siguientes afirmaciones son verdaderas o falsas? Coloque una marca de verificación en la columna correcta

Afirmación	Verdadero	Falso
En principio, una restricción check se puede agregar a una tabla que impida insertar cualquier dato	X	
Un atributo de llave primaria puede tener hasta un valor null		X
La restricción de llave primaria puede aumentarse con un on delete set null		X
La restricción on delete cascade definido para un atributo conlleva la eliminación de registros en otras tablas	X (Si se añade "cascade")	
La restricción check no puede alterar valores en una tabla	X	

Es bastante "abstracto" ...
Pero podría NO DEJAR
INSERTAR un valor si no
cumple ...

3. Considere la siguiente declaración para crear un esquema para una universidad:

CREATE TABLE student (id INT PRIMARY KEY, age INT NOT NULL);

CREATE TABLE lecture (id INT PRIMARY KEY, name VARCHAR (32) NOT NULL);

CREATE TABLE attends (sid INT, lid INT, constraint fk1 FOREIGN KEY (sid) REFERENCES student(id) ON DELETE CASCADE ON UPDATE CASCADE, constraint fk2 FOREIGN KEY (lid) REFERENCES lecture(id) ON DELETE NO ACTION ON UPDATE SET NULL)

1. Cuando se elimina un estudiante, se elimina el registro completo de "attends" claramente solo las filas que contengan ese referencia del estudiante ...
2. Cuando se actualiza la llave primaria de "students", se actualiza en "attends" para no dejar registros "huérfanos"
3. Cuando se elimina un registro de "lecture", la referencia en "attends" NO SE ELIMINA, y queda "obsoleta" (no hay existiría el registro referenciado)
4. Cuando se actualiza un "lecture", en "attends" la llave foránea pasa a ser nula ... (¿?)
5. Puede existir una tabla sin PRIMARY KEY y que permita (N <-> N)?

Inicialmente las tres tablas contienen la siguiente información:

student lecture attends

id	age
0	18
1	20
2	22

id	name
2	Databases
3	Literature
4	Biology

sid	lid
0	2
1	2
1	3
2	4
0	4

Complete la siguiente tabla indicando que tuplas contendrán las tablas después de ejecutar cada conjunto de declaraciones SQL.

SQL	student	lecture	attends
INSERT INTO lecture VALUES (1, 'Math I'); INSERT INTO lecture VALUES (2, 'Math II');		(1, 'Math'), (2, 'Databases'), (3, 'Literature'), (4, 'Biology') <i>// 2 da error, debe usarse UPDATE, ya existe registro con id = 2)</i>	
INSERT INTO student VALUES (9, 30); INSERT INTO attends VALUES (9, 2);	(0, 18), (1, 20), (2, 22), (9, 30)		(0, 2), (1, 2), (1, 3), (2, 4), (0, 4), (9, 2)
DELETE FROM lecture WHERE id>2; UPDATE lecture SET id=99 WHERE id<=2;		(99, 'Math'), <i>// Cambia error, PK constraint (99, 'Databases')</i>	(0, NULL), (1, <i>// ERROR PK</i>), (1, 3), (2, 4), (0, 4), (9, <i>// ERROR PK</i>) <i>// Por tanto no cambian datos</i>
DELETE FROM student WHERE age > (SELECT AVG(age) FROM student);	(0, 18), (1, 20), (2, 22)		(0, 2), (1, 2), (1, 3), (2, 4), (0, 4)

4. Considere los siguientes esquemas:

Schema A	Schema B	Schema C
CREATE TABLE tab1 (tab1_key INT NOT NULL); CREATE TABLE tab2 (tab2_ref INT NOT NULL);	CREATE TABLE tab1 (tab1_key INT NOT NULL UNIQUE); CREATE TABLE tab2 (tab2_ref INT NOT NULL, FOREIGN KEY (tab2_ref) REFERENCES tab1 (tab1_key));	CREATE TABLE tab1 (tab1_key INT NOT NULL UNIQUE); CREATE TABLE tab2 (tab2_ref INT NOT NULL, FOREIGN KEY (tab2_ref) REFERENCES tab1(tab1_key) ON DELETE NO ACTION UPDATE NO ACTION);

Schema D	Schema E
CREATE TABLE tab1 (tab1_key INT PRIMARY KEY); CREATE TABLE tab2 (tab2_key INT PRIMARY KEY, tab2_ref INT NOT NULL,	CREATE TABLE tab1 (tab1_key INT PRIMARY KEY); CREATE TABLE tab2 (tab2_key INT PRIMARY KEY, tab2_ref INT NOT NULL,

CONSTRAINT tab2_fk FOREIGN KEY (tab2_ref) REFERENCES tab1 (tab1_key) ON DELETE CASCADE ON UPDATE SET NULL); CREATE TABLE tab3 (tab3_ref INT, CONSTRAINT tab3_fk FOREIGN KEY (tab3_ref) REFERENCES tab2 (tab2_key) ON DELETE CASCADE ON UPDATE CASCADE);	FOREIGN KEY (tab2_ref) REFERENCES tab1 (tab1_key) ON DELETE CASCADE ON UPDATE CASCADE); CREATE TABLE tab3 (tab3_ref INT, FOREIGN KEY (tab3_ref) REFERENCES tab2 (tab2_key) ON DELETE SET NULL ON UPDATE CASCADE) ;
--	---

Las tablas inicialmente tienen el siguiente contenido:

Schemas A-C

tab1_key
1
2

tab2_ref
1
2

Schemas D-E

tab1_key
1
2

tab2_key	tab2_ref
10	1
20	2

tab3_key
10
20

¿Cuántos registros mantendrá cada esquema (sumados en todas sus tablas) después de ejecutar individualmente una de las siguientes sentencias? Complete la tabla siguiente y marque con 'X' un error de ejecución.

Sentencias

- I. UPDATE tab2 SET tab2_ref=3 WHERE tab2_ref=1;
- II. UPDATE tab1 SET tab1_key=3 WHERE tab1_key=1;
- III. DELETE FROM tab1 WHERE tab1_key=1;

	A	B	C	D	E
I					
II					
III					